

Q-Learning in Snake

By Hugo Garrido-Lestache, Vamsi Krishna, Theophanes Zinos

Abstract

We chose Q-Learning as our project due to its highly applicable and diverse nature. Q-Learning falls under the umbrella of machine learning, specifically reinforcement learning. It is well-suited for situations where human input can be significantly enhanced with minimal error through automation. Notably, this type of machine learning doesn't necessitate a large set of pre-labeled data, making it adaptable to scenarios with a "blank slate."

The initial implementation involved creating a Snake game from scratch using the PyGame library, with several modifications to tailor it for a Q-Learning environment. Once the Snake game was established, we introduced elements that directed the snake environment to run on a constant loop upon death, enabling autonomous data creation and gameplay. The final addition was the Learner environment, positioned above the Snake game cell. This component allowed the snake to save and retrieve data to and from a JSON file, maintaining a record of its past lives. Additionally, we defined four snake actions: up, down, left, and right, serving as descriptors for the snake's directional position.

As time permitted, we made some notable adjustments. For instance, we simplified and adjusted the states to prevent issues with the snake colliding with itself and causing premature termination. Another crucial modification involved tuning the hyperparameters to achieve a more optimal strategy.

We achieved our desired results by surpassing our goal of a score equal to or greater than 100, reaching a score of 108. This marked a significant milestone in the realm of reinforcement learning, indicating that with proper tuning and code design, the score could further improve.

While we accomplished our primary objectives, there were several potential additions to the program that time constraints prevented us from exploring. Implementing Deep Q-Learning could enhance the model's robustness and complexity, leading to improved overall results. Another potential future enhancement involves increasing the snake's state possibilities, enabling it to perceive the environment in more diverse ways. Further tuning of hyperparameters and fine-tuning the weights of the reward values could contribute to achieving even higher scores, making it a valuable addition to the program.

Introduction

When talking about the paradigms of Q-learning. It pops out as a potent and efficient algorithm enabling agents to learn the quality of actions within specific states of an environment. This learning mechanism is encapsulated within a Q-table which is a data structure that maps state-action pairs to rewards. Essentially, the Q-table guides the agent in selecting actions that maximize the cumulative reward.

Epsilon plays a crucial role in this learning process, acting as a parameter that looks after the trade-off between exploration and exploitation. With a high epsilon, the agent is encouraged to explore the environment, sampling a diverse array of actions. Whereas a lower epsilon guides the agent towards exploiting the known rewards in the Q-table.

In pursuit of demonstrating the efficiency of Q-learning, we established an objective score of 100 for our AI agent within the Snake game—a benchmark we found to be reasonable through the research we conducted. This score represents a challenging yet achievable target, pushing the boundaries of the agent's decision-making abilities while remaining attainable within a reasonable timeframe. The choice of Q-learning for this task is motivated by its proven track record in discrete, finite and simple environments, where it excels by iteratively updating the Q-table to hone the agent's policy based on the feedback loop of actions and rewards.

Baseline Model

In the initial phase of our project, we managed to establish a baseline model to set a standard for subsequent iterations and enhancements. The performance of this model served as the fundamental architecture helping us figure out the parameters of our Q-Learning algorithm. Upon evaluation, the model exhibited a decent level of proficiency in navigating the game grid and pursuing the apple. However, it was constrained by its inability to consistently avoid obstacles and achieve high scores, often failing to surpass the previous high score.

The limitations of the baseline model were primarily due to its base level understanding of the game dynamics. This was reflected in the simplistic hyperparameters that we set: a fixed reward for moving towards the apple, a bigger reward for getting the apple and a minor punishment for running into itself or the wall. It was also punished if it took too many steps without getting the apple. These rewards were not nuanced enough to teach the agent more complex strategies such as avoiding its tail or managing the increasing difficulty as the snake grew longer.

The hyperparameters chosen for this initial model included a low level of randomness (epsilon), which meant the agent rarely explored alternative paths that could potentially lead to worse long-term outcome. Coupled with a high learning rate, the agent's Q-table was updated in a aggressive manner that heavily favoured recent information, often at the cost of valuable past learning. As for the state-action space, it was designed to reflect a straightforward representation of the game, which, while computationally efficient, lacked the depth required for the agent to learn the intricacies of more advanced gameplay.

Model Improvements

Model 1

The initial enhancement to the model aimed to address the issue of the snake inadvertently colliding with itself. This problem arose from assigning the snake four possible actions at each state, a configuration that proved problematic when the snake's length exceeded 2. In such instances, taking any of these actions led to an immediate demise. Compounded with the epsilon factor, introducing a random action had a 25% chance of causing the snake's demise when its length surpassed 2, hindering its progress in achieving high scores. To rectify this, we streamlined the snake's available actions to turning right, turning left, and moving forward.

Furthermore, a decision was made to simplify the states, resulting in a reduction in the number of possible states and the size of the Q table. This simplification involved defining a state only based on the relative position of the food—whether to the right, left, or forward concerning the snake's orientation. The motivation behind this change was to increase the frequency of updates to individual Q table entries, given the reduced size of the table, ultimately leading to optimal values for each entry.

Despite these modifications, the model's performance worsened for several reasons. Firstly, the original model's low epsilon meant it seldom encountered issues with self-collision, rendering the preventive measures less impactful. Additionally, the decrease in the Q table's size led to excessive generalization when classifying specific game scenarios as states. This lack of granularity in information impeded the snake's ability to learn optimal actions for individual states. Nevertheless, the smaller table size accelerated the model's learning process.

In summary, these adjustments resulted in an overall score of 67.

Model 2

The second model specifically addressed a major issue in the first model, which was the overgeneralization of states. We opted to revert to the initial implementation and augment that model with additional information about its surroundings within each state. This adjustment was anticipated to result in more informed and consequently better actions. While this approach extended the learning time (time to reach an optimal score), it was expected to enable the model to achieve higher optimal scores compared to its predecessors. The extra information introduced to the model was the direction it was heading in during the previous turn.

Despite the relatively minimal change, the model exhibited improved performance compared to its predecessors and achieved a score of 75.

Final Model

For the final model, we sought to fine-tune hyperparameters in the pursuit of achieving an optimized version. While previous models prioritized swift training, the goal of this final iteration was to enhance performance even if it meant sacrificing some learning speed.

The first parameter under consideration was epsilon, which in prior models had been set exceptionally low at 0.1 for the initial 100 games and 0 thereafter. This low epsilon facilitated rapid learning but limited exploration. After experimenting with various ideas, we settled on initiating epsilon at 1 and gradually reducing it exponentially by multiplying it by 0.992 after each game. This approach allowed extensive exploration of behaviours early in the training process and accumulated experience in high-scoring scenarios during the later stages.

The second set of parameters adjusted included the learning rate and discount decay. Initially set at a high value to expedite learning, the learning rate was scaled down to accommodate the increased number of games played. A lower learning rate encouraged less aggressive alterations to more Q values, facilitating quicker strategy selections. The discount decay, which determines the significance of future rewards in comparison to current ones, was set close to 1 to encourage the snake to adopt a long-term strategy, discouraging behaviours that prioritize immediate rewards at the expense of survival.

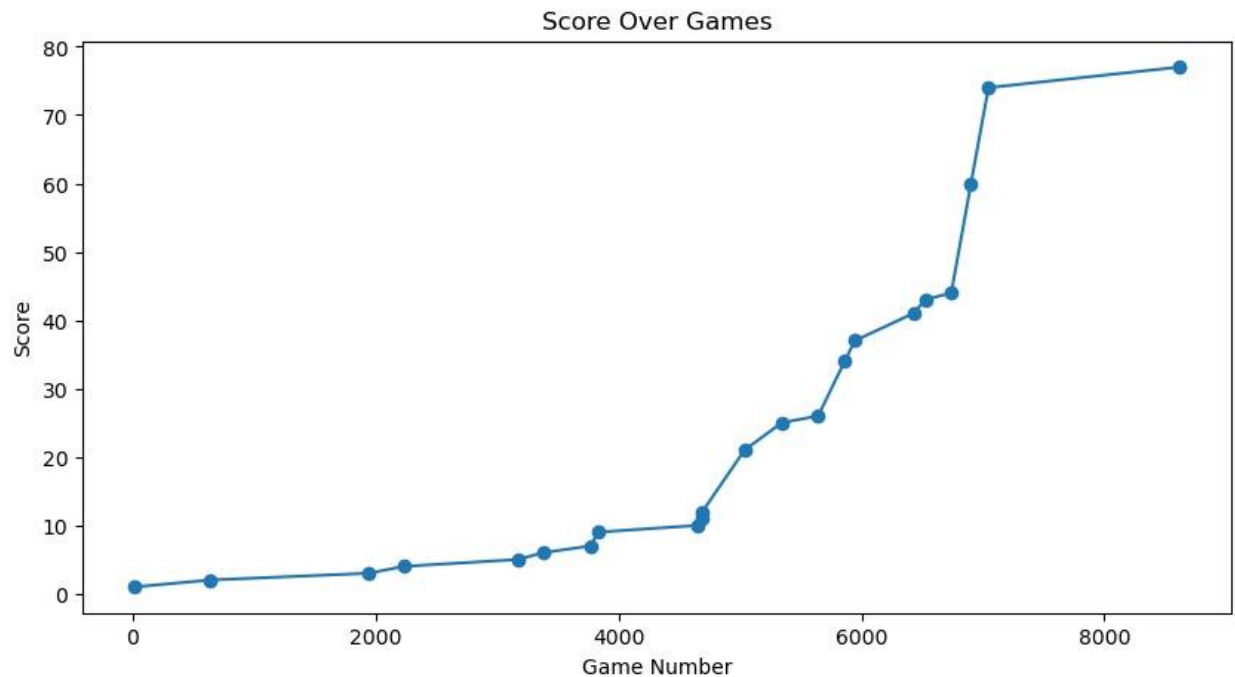
The last parameter modified was the reward system. In earlier models, the snake received substantial rewards for moving towards the apple's location, promoting rapid learning. However, this strategy could lead to suboptimal behaviours, especially when the snake grew longer and its tail blocked the direct path to the apple. To address this, the reward for moving toward the apple was significantly reduced. This adjustment allowed the model to focus on obtaining apples rather than pursuing them directly. Additionally, maintaining a small reward for moving toward the apple in the first game encouraged the initial exploration but did not unduly influence later games.

Furthermore, rewards for dying were increased to act as a stronger deterrent, reinforcing the lesson that collisions with walls or the snake's own tail were undesirable outcomes. This adjustment aimed to solidify the model's understanding that such actions were not beneficial in the pursuit of optimal gameplay.

Results

After letting the model train for over 10,000 games and then letting it play for another 10,000 games to try and get a high score, the model got a score of 108 which helped us overcome our goal.

Below is the graph showcasing the progressive high scores through it learning: [Does not include our best high score run]



This graph depicts the progression of scores as the AI plays through successive games, we can deduce the trend that begins with a slow but steady increase in scores, characteristic of an exponential growth pattern. This initial phase reflects the influence of a higher epsilon value, which prioritizes exploration over exploitation. The AI's strategy during this phase is to explore the game environment widely, trying out various actions to understand their consequences, which naturally results in a slower increase in score as the AI is not consistently making the most optimal moves.

As the AI continues to learn and store experience and as the impact of the epsilon diminishes, the AI bring to truly develop. This transition is marked by the AI's adoption of a more optimal strategy, which results in a faster increase in scores. The AI begins to refine its approach, making more informed decisions that get higher rewards.

Interestingly, towards the end of the graph, there is a noticeable decline in score. This could be attributed to the AI potentially reaching the limits of its current strategy, which has been optimized within the confines of its learned experience. As the epsilon value lowers further, the AI's behaviour for exploration decreases, and without new exploratory actions, it might repeatedly encounter and fail to navigate particular scenarios that it has not yet learned to overcome. This plateau and subsequent dip could signal the need to reintroduce a degree of exploration to surpass this newfound ceiling, highlighting the balance between exploration and exploitation in reinforcement learning.

Conclusion

The takeaway we had from this presentation was that Q-Learning must be built in a very robust manner, as it requires an environment that appropriately addresses all possibilities and situations within a given scenario. We did well in having our program run quite fast and produce lots of data within minutes. This allowed us to gather lots and lots of data for each modification made on the program. We also wrote the code in a manner that can be read and used in a “fairly” different scenario so others could make changes and try out other games. The code also serves a certain replicability in that any developer could take the code and get the program to run with the same environment with an end goal similar behaviour. It is important to note however that the nature of Q-learning and implementation of Randomness disables the ability to reproduce the same data twice. We also had a good sense of score ceiling that the snake was unable to exceed as well as a very similar learning rate throughout all test runs.

If we could change certain elements within this program, there would be a multitude of options. One such option that would work well would be to increase the state information for the q table. By increasing the total states, we could add further context for it to learn from. This would be a good addition as the snake would be able to calculate more complex routing as well as preventing it from dying to itself significantly more. Another good change to make in the future would be to tune the parameters to a calculated “best”. Although tuning tends to be more of an art, we did not use complex mathematics at all to find best fit parameters. The current weightings placed were much more logically thought out for how we wanted the snake to behave. This would allow both a better score ceiling and a more consistent performance from the snake. Another improvement that could be made would be to implement Deep Q-Learning (DQL), in contrast to Vanilla Q-learning. The reasons that this would be a better model type is that we would be able to give the model much more information, which would then yield better results. The snake would have a general better game sense allowing it to make more informed and better actions. DQL takes a lot of careful work, and we thought it would be a better foundation to learn from by using Machine Learning first.